

第一次作业解答

姓名: 郑文翔 学号: 25S003050

2026 年 4 月 13 日

1. 基本概念解释

- **NP 问题**: 指其解可以在多项式时间内验证的问题。即若给出一个候选解, 我们可以在多项式时间内检查其是否正确。
- **NP-hard 问题**: 若任意 NP 问题都能在多项式时间内归约到问题 H , 则称 H 为 NP-hard。它表示 H 至少与 NP 中最难的问题一样难, 但 H 不一定属于 NP。
- **NPC 问题**: 既属于 NP, 又是 NP-hard 的问题称为 NP 完全问题 (NPC)。它们是 NP 中“最难”的那一类问题。若某个 NPC 问题能在多项式时间内求解, 则有 $P = NP$ 。

- **渐进符号**:

- $O(g(n))$: 渐进上界。存在常数 $c > 0, n_0 > 0$, 当 $n \geq n_0$ 时, $0 \leq f(n) \leq cg(n)$ 。
- $o(g(n))$: 严格渐进上界。对任意常数 $c > 0$, 存在 $n_0 > 0$, 当 $n \geq n_0$ 时, $0 \leq f(n) < cg(n)$ 。
- $\Omega(g(n))$: 渐进下界。存在常数 $c > 0, n_0 > 0$, 当 $n \geq n_0$ 时, $0 \leq cg(n) \leq f(n)$ 。
- $\Theta(g(n))$: 渐进确界, 同时满足 $f(n) = O(g(n))$ 和 $f(n) = \Omega(g(n))$ 。

- **时间复杂度**: 算法执行的基本操作次数关于输入规模 n 的渐进增长率。
- **空间复杂度**: 算法执行过程中除输入本身外额外占用存储空间关于输入规模 n 的渐进增长率。

- **函数阶的性质**:

- 若 $f(n) = \Theta(g(n))$, 则 $g(n) = \Theta(f(n))$;
- 若 $f(n) = O(g(n))$ 且 $g(n) = O(h(n))$, 则 $f(n) = O(h(n))$;
- 若 $f_1(n) = O(g_1(n))$, $f_2(n) = O(g_2(n))$, 则

$$f_1(n) + f_2(n) = O(\max\{g_1(n), g_2(n)\}),$$

且当各函数最终非负时,

$$f_1(n) + f_2(n) = \Theta(\max\{f_1(n), f_2(n)\}).$$

- 多项式中最高次项决定函数阶, 例如

$$3n^3 + 5n^2 + 7 = \Theta(n^3).$$

2. 证明或反例

题目：证明或举出反例： $f(n) + g(n) = \Theta(\max(f(n), g(n)))$

证明. 该结论在 $f(n), g(n)$ **最终非负**时成立。

设

$$M(n) = \max(f(n), g(n)).$$

由于 $f(n) \leq M(n)$ 且 $g(n) \leq M(n)$, 所以

$$f(n) + g(n) \leq 2M(n).$$

另一方面, 由于 $f(n), g(n) \geq 0$, 故

$$f(n) + g(n) \geq M(n).$$

于是存在常数 $c_1 = 1, c_2 = 2$, 使得当 n 充分大时,

$$c_1 M(n) \leq f(n) + g(n) \leq c_2 M(n).$$

因此

$$f(n) + g(n) = \Theta(\max(f(n), g(n))).$$

注意：若不要求最终非负，该结论不一定成立。

□

3. 求解递归方程

$$T(n) = 4T\left(\frac{n}{16}\right) + \sqrt{n}$$

解答：用主定理。这里

$$a = 4, \quad b = 16, \quad f(n) = n^{1/2}.$$

计算

$$n^{\log_b a} = n^{\log_{16} 4} = n^{1/2}.$$

故

$$f(n) = \Theta(n^{\log_b a}).$$

属于主定理第二种情况，因此

$$T(n) = \Theta\left(n^{1/2} \log n\right).$$

4. 求解递归方程

$$T(n) = 4T(n-1) + 2^n + 2T\left(\frac{n}{2} - 1\right)$$

解答：由于各项非负，先得下界

$$T(n) \geq 4T(n-1) \geq 4^2T(n-2) \geq \cdots \geq 4^nT(0),$$

故

$$T(n) = \Omega(4^n).$$

下面证明上界。设对所有小于 n 的正整数 m ，有

$$T(m) \leq C4^m$$

成立。则

$$\begin{aligned} T(n) &= 4T(n-1) + 2^n + 2T\left(\frac{n}{2} - 1\right) \\ &\leq 4 \cdot C4^{n-1} + 2^n + 2 \cdot C4^{n/2-1} = C4^n + 2^n + \frac{C}{2} \cdot 2^n. \end{aligned}$$

因为 $2^n = o(4^n)$ ，所以当 n 足够大时，

$$2^n + \frac{C}{2}2^n \leq C4^n,$$

从而

$$T(n) \leq 2C4^n.$$

因此

$$T(n) = O(4^n).$$

综上，

$$T(n) = \Theta(4^n).$$

5. 求解递归方程

$$T(n) = 5T(n-2) + \log n$$

解答：先看齐次部分

$$T(n) = 5T(n-2).$$

设解为 $T(n) = r^n$ ，则有特征方程

$$r^2 - 5 = 0,$$

解得

$$r = \pm\sqrt{5}.$$

故齐次解为

$$T_h(n) = C_1(\sqrt{5})^n + C_2(-\sqrt{5})^n.$$

因此其主阶为

$$T_h(n) = \Theta((\sqrt{5})^n) = \Theta(5^{n/2}).$$

非齐次项 $\log n$ 仅为多项式对数级，增长速度远慢于指数函数，不改变最高阶。因此

$$T(n) = \Theta(5^{n/2}).$$

6. 求解递归方程

$$T(n) = T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + T\left(\left\lfloor \frac{n}{4} \right\rfloor\right) + \log n$$

解答：用 Akra-Bazzi 定理。设

$$T(n) = \sum_{i=1}^k a_i T(b_i n) + g(n),$$

其中

$$a_1 = a_2 = 1, \quad b_1 = \frac{1}{2}, \quad b_2 = \frac{1}{4}, \quad g(n) = \log n.$$

求 p 满足

$$\left(\frac{1}{2}\right)^p + \left(\frac{1}{4}\right)^p = 1.$$

令 $x = (1/2)^p$ ，则

$$x + x^2 = 1.$$

解得正根

$$x = \frac{\sqrt{5} - 1}{2}.$$

故

$$p = \log_2 \left(\frac{2}{\sqrt{5} - 1} \right) = \log_2 \left(\frac{\sqrt{5} + 1}{2} \right).$$

因为 $g(n) = \log n = O(n^{p-\varepsilon})$ (取某个足够小的 $\varepsilon > 0$ 即可)，所以

$$T(n) = \Theta(n^p) = \Theta\left(n^{\log_2 \frac{\sqrt{5}+1}{2}}\right).$$

7. 从 0 开始打印到最大的 n 位正整数

问题分析：由于 n 可能很大，不能直接用整型保存数值，应使用长度为 n 的字符数组模拟十进制数。题目要求从 0 一直打印到最大的 n 位数 $10^n - 1$ 。

方法一（递归枚举每一位）：递归地给每一位赋值 $0 \sim 9$ ，最后输出时去除前导零；若全为 0，则输出 0。

Algorithm 1: PrintNumbers(n)

Input: 整数 n
if $n \leq 0$ **then**
 $\perp$ **return**
 $res \leftarrow$ 长度为 n 的字符数组;
for $i \leftarrow 0$ **to** $n - 1$ **do**
 $\perp$ $res[i] \leftarrow '0'$;
DFS(0);

Function DFS(idx):
if $idx = n$ **then**
 PrintNumber(res);
 $\perp$ **return**;
for $d \leftarrow 0$ **to** 9 **do**
 $\perp$ $res[idx] \leftarrow \text{char}(d + '0')$;
 DFS($idx + 1$);

PrintNumber 的规则：跳过前导零；如果整个数组全为 0，则输出一个字符 0。

复杂度分析：共有 10^n 个状态，每次输出最多需 $O(n)$ 时间，因此总时间复杂度为 $O(n \cdot 10^n)$ ；递归栈和数组空间为 $O(n)$ 。

8. 友谊点对统计

题意理解：若存在一条边平行于坐标轴的矩形，其内部和边界上恰好只包含点集 S 中的两个点 p_i, p_j ，则称它们为“友谊点对”。这不是最近点对问题，关键在于矩形内不能有第三个点。

分治思路：将点按 x 坐标排序，从中间位置分成左半部分 S_L 和右半部分 S_R 。

递归统计：

$$ans = ans_L + ans_R + ans_{cross},$$

其中前两项分别为完全位于左右子集中的友谊点对数。

难点在于统计跨越中线的友谊点对。对于跨边界点对 (p, q) ，设它们构成的轴对齐最小包围矩形为 $R(p, q)$ 。要成为友谊点对，必须满足矩形中不存在第三个点。于是跨边界统计问题可转化为：

枚举跨越分界线的候选点对，并判断由这两个点确定的矩形内部是否为空。

一种可行做法是：在分治每一层中，将涉及跨边界的点按 y 坐标排序，并结合二维前缀计数/树状数组，支持快速查询某个轴对齐矩形内点数。若矩形内点数恰为 2，则该点对为友谊点对。

复杂度：若每层分治中借助数据结构在 $O(\log n)$ 时间内完成一次矩形计数查询，则整层代价可控制在近线性或线性对数级，总复杂度可写为

$$T(n) = 2T(n/2) + O(n \log n),$$

因此总时间复杂度为

$$O(n \log^2 n).$$

说明：本题核心不是距离，而是“由两点确定的轴对齐矩形中没有第三点”。

9. 二叉树顶点对距离 $\leq \alpha$

分治思路：由于输入是**加权二叉树**，可直接利用树的左右子树结构进行分治。
设当前根为 r ，左子树为 L ，右子树为 R 。满足条件的点对分三类：

- 两点都在左子树中；
- 两点都在右子树中；
- 一点在左子树（或为根），另一点在右子树（或为根）。

前两类可递归求解。关键是第三类。对当前根 r ：

- 计算左子树中所有点到 r 的距离，记为数组 A ；
- 计算右子树中所有点到 r 的距离，记为数组 B ；
- 分别排序后，用双指针统计满足

$$A[i] + B[j] \leq \alpha$$

的跨子树点对个数；

- 还需统计根与左右子树中各点形成的合法点对。

Algorithm 2: CountPairs($root, \alpha$)

Input: 二叉树根结点 $root$ ，阈值 α

if $root = null$ **then**

return 0

$ans \leftarrow \text{CountPairs}(root.left, \alpha) + \text{CountPairs}(root.right, \alpha);$

$A \leftarrow$ 左子树所有结点到 $root$ 的距离;

$B \leftarrow$ 右子树所有结点到 $root$ 的距离;

对 A, B 升序排序;

$ans \leftarrow ans + \text{CrossCount}(A, B, \alpha);$

统计 $root$ 与左右子树中距离不超过 α 的结点数并加入 ans ;

return ans ;

复杂度：设每个结点在其所在子问题中被收集并排序一次，则递推式可写为

$$T(n) = T(n_L) + T(n_R) + O(n \log n).$$

若树平衡，则总复杂度为 $O(n \log^2 n)$ ；若进一步优化排序传递，可降为 $O(n \log n)$ 。

10. 文字机器最快完成时间

题意修正：机器不是从 0 个字开始，而是从 1 个字开始。

逆向思考：从长度 n 倒推到 1：

- 若当前长度为偶数，则上一步最优一定是由长度 $n/2$ 复读得到；
- 若当前长度为奇数，则最后一步只能是“写字”，即从 $n-1$ 到 n 。

因此设 $f(n)$ 为从 1 个字扩展到 n 个字的最少时间，则

$$f(1) = 0,$$
$$f(n) = \begin{cases} f(n/2) + 1, & n \text{ 为偶数}, \\ f(n-1) + 1, & n \text{ 为奇数}. \end{cases}$$

复杂度：每一步都使规模减少至少一半或减 1，故时间复杂度为 $O(\log n)$ ，空间复杂度为 $O(1)$ （若迭代实现）。

11. 回文子串统计与最长回文子串

算法：中心扩展法。

每个回文串都有一个“中心”。中心可能是一个字符（奇数长度回文），也可能是两个字符之间的空隙（偶数长度回文）。一共共有 $2n-1$ 个中心。

Algorithm 3: CountAndLongestPalindrome(S)

Input: 字符串 S ，长度为 n

$count \leftarrow 0$, $bestL \leftarrow 0$, $bestLen \leftarrow 0$;

for $c \leftarrow 0$ **to** $2n-2$ **do**

$left \leftarrow \lfloor c/2 \rfloor$;

$right \leftarrow left + (c \bmod 2)$;

while $left \geq 0$ **and** $right < n$ **and** $S[left] = S[right]$ **do**

$count \leftarrow count + 1$;

if $right - left + 1 > bestLen$ **then**

$bestLen \leftarrow right - left + 1$;

$bestL \leftarrow left$;

$left \leftarrow left - 1$;

$right \leftarrow right + 1$;

return ($count, S[bestL \dots bestL + bestLen - 1]$);

复杂度：时间复杂度 $O(n^2)$ ，空间复杂度 $O(1)$ 。

12. 询问区间中随机子区间的期望

题意：共有 M 次询问。每次给定区间 $[L, R]$ ，在该区间内均匀随机选择一个子区间 $[x, y]$ ，求

$$\mathbb{E} \left[\frac{\max(a[x..y]) + \min(a[x..y])}{2} \right].$$

设当前询问区间长度为

$$len = R - L + 1,$$

则其中子区间总数为

$$K = \frac{len(len + 1)}{2}.$$

于是期望为

$$E = \frac{\sum \max + \sum \min}{2K},$$

其中求和范围均为区间 $[L, R]$ 内所有连续子区间。

单次询问的求法：对固定询问 $[L, R]$ ，取子数组 $b = a[L..R]$ 。利用单调栈分别计算：

- 每个元素作为子区间最大值的次数；
- 每个元素作为子区间最小值的次数。

若元素 $b[i]$ 左侧第一个严格大于它的位置为 $preGreater[i]$ ，右侧第一个大于等于它的位置为 $nextGreaterEq[i]$ ，则它作为最大值的贡献次数为

$$(i - preGreater[i]) \cdot (nextGreaterEq[i] - i).$$

类似地可求它作为最小值的次数。

于是

$$\sum \max = \sum_i b[i] \cdot cntMax[i], \quad \sum \min = \sum_i b[i] \cdot cntMin[i].$$

最终

$$E = \frac{\sum \max + \sum \min}{2K}.$$

复杂度：单次询问若直接在子数组上做单调栈，时间复杂度为 $O(R - L + 1)$ ，空间复杂度为 $O(R - L + 1)$ 。因此总复杂度为

$$O \left(\sum_{q=1}^M |R_q - L_q + 1| \right).$$

若需进一步优化到更快的多询问复杂度，则需更复杂的数据结构，本题写到这里已能完整回答基本算法思想。

13. 盒子堆叠问题（动态规划）

(1) 优化子结构

若某种摆放方式的盒子 i 放在最上层，则其下方一定是一个“底面长宽都严格大于 i 的盒子所构成的最优堆叠”。因此整体最优解可以由更小规模的最优子问题组合得到，具有最优子结构。

(2) 子问题重叠性

不同堆叠方案在转移时会多次询问“以某个盒子朝向作为顶层时的最大高度”，因此这些子问题会被重复访问，具有重叠子问题性质，适合用动态规划。

(3) 代价递归方程

每个盒子 (l_i, w_i, h_i) 可旋转，产生 3 种朝向。记某一朝向的底面为 (L_i, W_i) ，高为 H_i ，并约定 $L_i \geq W_i$ 。

设

$$dp[i] = \text{以第 } i \text{ 个朝向盒子作为最顶层时的最大总高度.}$$

则有

$$dp[i] = H_i + \max\{dp[j] \mid L_j > L_i, W_j > W_i\}.$$

若不存在这样的 j ，则

$$dp[i] = H_i.$$

由于题目允许多次使用盒子，若无限重复且存在可循环堆叠的尺寸链，则高度可能无上界。通常默认“尺寸严格递减”使得每次往上放置时底面严格变小，因此不会出现无限循环；在此条件下，动态规划仍成立。

(4) 算法描述

1. 对每个盒子生成 3 种朝向；
2. 统一令底面较长边记为 L ，较短边记为 W ；
3. 按底面积或按 (L, W) 字典序从大到小排序；
4. 进行类似 LIS 的动态规划：

$$dp[i] = H_i + \max\{dp[j] \mid j < i, L_j > L_i, W_j > W_i\}.$$

5. 最终答案为

$$\max_i dp[i].$$

(5) 时间复杂度和空间复杂度

设共有 $3n$ 个朝向，则排序复杂度为 $O(n \log n)$ ，动态规划双重循环复杂度为 $O(n^2)$ ，故总时间复杂度为

$$O(n^2).$$

空间复杂度为

$$O(n).$$

14. 股票买卖 II

算法描述：由于可以进行任意多次交易，且任意时刻最多持有一股，因此只要今天到明天价格上涨，就把这段上涨收益全部拿到即可。

Algorithm 4: MaxProfit(prices)

Input: 价格数组 $prices$

$profit \leftarrow 0$;

for $i \leftarrow 1$ **to** $prices.length - 1$ **do**

if $prices[i] > prices[i - 1]$ **then**

$profit \leftarrow profit + (prices[i] - prices[i - 1]);$

return $profit$;

复杂度：时间复杂度 $O(n)$ ，空间复杂度 $O(1)$ 。

15. 分治法求最大子数组和

思路：最大子数组和只可能出现在以下三种情况之一：

- 完全位于左半部分；
- 完全位于右半部分；
- 横跨中点。

因此对区间 $[l, r]$ 分治求解，最后返回三者最大值。

Algorithm 5: MaxSubArray(A, l, r)

Input: 数组 A 与区间 $[l, r]$

if $l = r$ **then**

return $A[l]$;

$mid \leftarrow \lfloor (l + r) / 2 \rfloor$;

$leftAns \leftarrow \text{MaxSubArray}(A, l, mid)$;

$rightAns \leftarrow \text{MaxSubArray}(A, mid + 1, r)$;

$sum \leftarrow 0, leftBest \leftarrow -\infty$;

for $i \leftarrow mid$ **do**

$sum \leftarrow sum + A[i]$;

$leftBest \leftarrow \max(leftBest, sum)$;

$sum \leftarrow 0, rightBest \leftarrow -\infty$;

for $i \leftarrow mid + 1$ **to** r **do**

$sum \leftarrow sum + A[i]$;

$rightBest \leftarrow \max(rightBest, sum)$;

$crossAns \leftarrow leftBest + rightBest$;

return $\max(leftAns, rightAns, crossAns)$;

复杂度：递推式为

$$T(n) = 2T(n/2) + O(n),$$

由主定理可得

$$T(n) = O(n \log n).$$

16. 分治与动态规划的区别

- **共同点：**二者都通过把原问题拆成规模更小的子问题来求解。
- **分治法适用的问题：**适合子问题之间**相互独立、几乎不重叠**的问题。例如归并排序、快速排序、最大子数组和、最近点对等。典型模式是“分解—递归求解—合并”。
- **动态规划适用的问题：**适合具有**最优子结构**和**重叠子问题**的问题。例如背包问题、最长公共子序列、矩阵连乘、盒子堆叠等。其核心是避免重复计算，常用记忆化搜索或自底向上表格法。
- **关键区别：**
 - 分治法：子问题通常彼此独立；
 - 动态规划：子问题往往大量重叠，因此要“记住”已经求出的结果。